

* Driver connect contain the implementation classes of JDBC API (.jar file)

* Standard step followed for developing JDBC Application.

- 1) Load and register the driver
- 2) Establish the connection b/w java application and database
- 3) create a Statement object
- 4) Send and execute the query
- 5) process the result for ResultSet
- 6) close the connection.

Step 1 Load and register driver.

A third party db vendor class which implement `java.sql.Driver` (I) is called as Driver.

This class object we need to create and register it with JRE to set up JDBC environment to run jdbc application.

```
public class com.mysql.cj.jdbc.Driver extends
    com.mysql.cj.jdbc.NonRegisteringDriver
    implement java.sql.Driver {
```

```
    public com.mysql.cj.jdbc.Driver() throw SQLException;
    static { }
}
```

MySQL JAR, Driver class is implementing `java.sql.Driver`, so Driver class object should be created and it should be registered to set up the JDBC environment inside JRE.

2. Establish the connection b/w java application and database

`public static Connection getConnection (String url, String username, String password)` throws `SQLException`.

`public static Connection getConnection (String url, Properties) throws SQLException`.

`public static Connection getConnection (String url)`

The below create the object of Connection interface.

```
Connection con = DriverManager.getConnection (url, username, password);
```

`getConnection()` create an object of class which implements `Connection (I)` & that class object is collected by `Connection (I)`. This feature in java

refers to a) Abstraction
b) Polymorphism.

3. create a ~~Sote~~ Statement object

```
public abstract Statement createStatement()  
public abstract Statement createStatement( int, int)  
public abstract Statement createStatement( int, int, int)
```

```
Statement statement = con. createStatement();
```

4. Send and execute the Query

According to java developer perspective,
we categorise queries into 2 types

- a) Select Query
- b) Nonselect Query

Method for executing the Query are

a. executeQuery() : For select query
we use this method.

b. executeUpdate() : For insert, update
delete query.

c. execute() : For both select and
non select query.

```
public abstract ResultSet executeQuery( String query)
```

```
throw SQLException
```

@sahul-barake

```
ResultSet resultSet = Statement.executeQuery  
("Select * From Student");
```

5) process the resultSet

```
public abstract boolean next() throws SQLException
```

This method check whether the next record is available or not return true if available otherwise false

```
while(resultSet.next()) { }
```

6) close the connection

Example code :

```
import java.sql.*;  
import com.mysql.cj.jdbc.Driver;
```

```
class TestApp { throws SQLException
```

```
public static void main (String args[]) {
```

```
//1. load and register the driver
```

```
Driver d = new Driver();
```

```
DriverManager.registerDriver(d);
```

//2. Establish the connection

```
String url = "jdbc:mysql://localhost:3306/db";  
String user = "root";  
String password = "root";
```

```
Connection con = DriverManager.getConnection(url, user, pass);
```

//3. create the statement object to send the data.

```
Statement stm = con.createStatement();
```

//4. Send and execute the query

```
String sqlquery = "select * from student";
```

```
ResultSet res = stm.executeQuery(sqlquery);
```

// 5. process the result set

```
while(res.next()) {
```

(1)

```
String name = res.getString('Name');
```

```
Integer rollno = res.getInt('RollNo');
```

```
S.o.p(name + "\t" + rollno);
```

```
}
```

// 6. close the connection.

```
con.close();
```

In earlier version of jdbc 3.x we are loading and registering the driver using the following approach.

```
Driver d = new Driver();
DriverManager.registerDriver(d);
```

Alternate to this as;

```
Class.forName("com.mysql.cj.jdbc.Driver");
```

* We say a class represent a Driver, iff the class has implemented an interface `java.sql.Driver (I)`

MySQL => `Driver(C)` implements `Driver(I)`

Oracle => `OracleDriver(C)` implements `Driver(I)`

PostgreSQL => `PostgreSQLDriver(C)` implements `Driver(I)`

* From JDBC 4.x onwards loading and registering would happen automatically depending upon the jar added in the classpath location of the project.

* Formatting the string query to accept the dynamic input.

```
int a sname = " " + sname + " ";
```

```
"insert into student('sname', 'sage') values  
(" + sname, " + sage + ")";
```

To resolve the problem we use
format method of string class.

Format specifier for string '%s'

for integer %d

for float %f.

```
String.format("insert into student ('sname','age')  
values ('%s', '%d')", sname, age);
```

* Problem with the statement
object

```
Statement stmt = con.createStatement();  
ResultSet rs = stmt.executeQuery("select *  
from student");
```

If we use the statement object,
same query will be compiled every
time and the query should be
executed, this create performance
problem

e.g. IRCTC APP(select query), BMS APP(select
query)

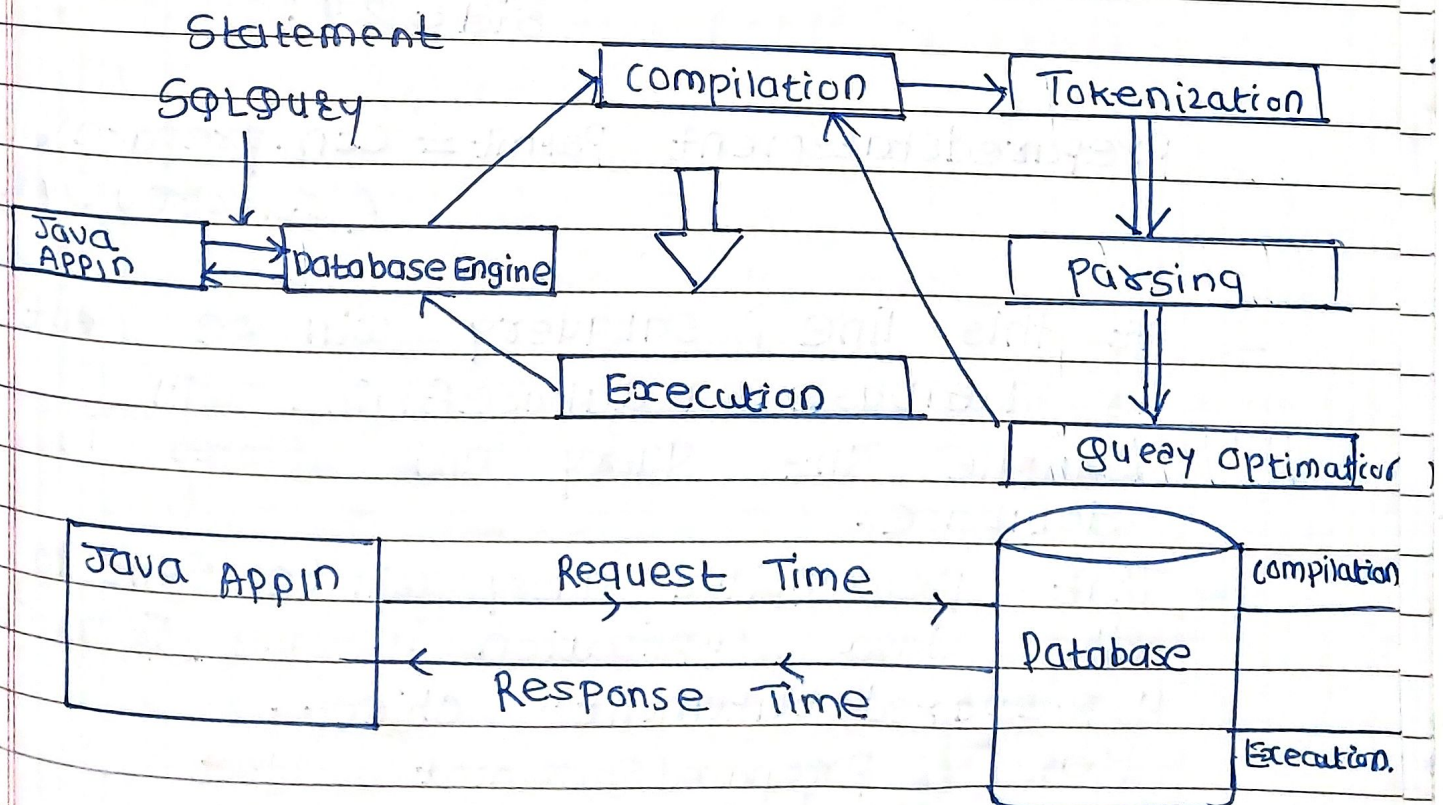
* PreparedStatement Object

To resolve the above problem, don't use statement object, use "PreparedStatement" (pre-compiled statement).

In case of PreparedStatement, the query will be compiled only once even though we are executing it multiple times with change or no change in input. This would overall increase the performance.

Signature:

```
public PreparedStatement prepareStatement (String query)
    throws SQLException.
```



Sometime in our application, we need same query but that query has to be executed multiple time with change in input.

IRCTC App^o: book train ticket.

e.g:- `Select * from trains where Source='xxx'`
and `destination = 'yyy'`

// creating a precompiled query which is used at the runtime to execute with the value

```
String selectQuery = " select sid, sname  
From Student where  
sid = ? ";
```

```
PreparedStatement pstmt = con.prepareStatement  
(selectQuery);
```

- At this line, `sqlQuery` will be sent to database; `DatabaseEngine` will compile the query and stores in database.

That precompiled query will be sent to the Java application in the form of " `PreparedStatement` " object.

Hence `PreparedStatement` object is called " `precompiled query` " object.

// Execute the precompiled query by setting the inputs.

```
Integer sid = 10;
```

```
pstmt.setInt(1, sid);
```

pos.

```
ResultSet resultSet = pstmt.executeQuery();
```

// whenever we execute methods, database engine will not compile query once again and it will directly execute that query, so that overall performance will be improved.

e.g:- `String sqlquery = "insert into student ('sid', 'sname') values (?, ?);"`

```
PreparedStatement pstmt = con.prepareStatement(sqlquery);
```

```
pstmt.setInt(1, 10);
```

```
pstmt.setString(2, "Rahul");
```

```
int rowcnt = pstmt.executeUpdate();
```

* SQL Injection : by changing the SQL query the person can access

the data from database unauthorizedly is called SQL injection.

Consider the below query

e.g: `select * from users where username = ' ' + Uname + ' ' and
upwd = ' ' + upwd + ' ' ;`

`username = 'Rahul' --`

`password = '1234'`

Here if the user enter wrong password still he get access of data because "--" in sql is treated as singleline comment.

* In Statement object to send query, then the problem of SQL Injection will happen.

e.g:- `"select count(*) from user where username = ' ' + uname + ' ' and upswd = ' ' + upwd + ' ' ;`

Result If we execute it will execute like.



`select count(*) from users where
username = ' ' + Sachin' -- " ;`

* If we use PreparedStatement Object to send query, then the problem of SQL Injection will not happen. because in ps it is treated as value. not comment.